

Лабораторна 8

Впізнавання місць (DBoW), Docker і YOLO

Гайд простою мовою: що, навіщо і як

Як читати цей гайд:

- спершу — **суть і мета** лаби на одній сторінці;
- кожна тема йде **шарами**: спершу «Простими словами» (аналогія, без формул), потім «Як це працює» (кроки), потім «Глибше» (формули, числа, деталі реалізації);
- якщо щось незрозуміло в «Глибше» — поверніться до простого шару: він пояснює ту саму ідею;
- для кожного питання завдання — ключові думки й приклади, з яких ви пишете **свою** відповідь;
- команди для YOLO перевірені в Docker-образі курсу, приклади результатів — справжні.

Зміст

1	Суть і мета лаби	3
1.1	Одним абзацом	3
1.2	Цілі (чого маєте навчитися)	3
1.3	Що здавати	3
2	Deliverable 1 — мішок візуальних слів (DBoW)	5
2.1	Навіщо це взагалі: проблема впізнавання місця	5
2.2	Головна ідея: картинка як текст	5
2.3	Як це працює: крок за кроком (як у статті)	6
2.4	Питання 1 — стійкість до освітлення і ракурсу	8
2.5	Питання 2 — навіщо IDF	8
2.6	Питання 3 — розмір словника	9
3	Deliverable 2 — Docker	11
3.1	Простими словами	11
3.2	Встановлення і скриншот	12
3.3	Думки для відповідей	12
4	Deliverable 3 — YOLO з Ultralytics	13
4.1	Що таке YOLO, простими словами	13
4.2	П'ять задач	13
4.3	Як вибрати фото (найчастіша причина втрачених балів)	15
4.4	Встановлення і запуск	15
4.5	Бонус: офіційний Docker-образ Ultralytics	16
5	Міст до Lab 8.5: пакет <code>lab8</code> у репозиторії	17
6	Чекліст здачі	18
A	Словничок	19

1 Суть і мета лаби

1.1 Одним абзацом

У лабах 5–7 ми вчили робота рахувати, **як він рухається**: знаходити ключові точки, оцінювати рух між кадрами, згладжувати траєкторію GTSAM-ом. Але в такого робота є дві сліпі плями. По-перше, похибки накопичуються: пройшовши коло, він не знає, що повернувся туди, звідки почав (ми бачили це в Lab 7 — без MoCap траєкторія «пливе»). По-друге, він бачить лише точки, а не **речі**: для нього немає «стілця» чи «людини». Lab 8 закриває обидві плями і додає інструмент, щоб усе це можна було запустити на будь-якому комп'ютері:

Тема	Яку проблему розв'язує	Де знадобиться далі
DBoW (мішок візуальних слів)	Робот швидко впізнає: «я тут уже був». Це дає замикання петлі (loop closure), що прибирає накопичений дрейф.	Lab 9: ORB-SLAM3 і Kimera використовують саме DBoW2 для loop closure.
Docker	«У мене працює, а в тебе ні» — Docker пакує код разом з усіма бібліотеками в одну «коробку».	Lab 9 запускається лише в Docker; уся наша робота з ROS 2 теж іде в Docker-образі.
YOLO	Нейромережа за один прохід знаходить на фото об'єкти: що це і де воно.	Lab 8.5: YOLO знаходить ведмедика у відео, GTSAM рахує, де він стоїть у 3D.

1.2 Цілі (чого маєте навчитися)

1. **Розуміти DBoW**: як з картинки зробити «текст із візуальних слів» і як за ним миттєво знайти схожу картинку серед тисяч.
2. **Прочитати класичну статтю** (DBoW, 2012) і пояснити її ключові рішення своїми словами.
3. **Знати базові поняття Docker**: Dockerfile, образ, контейнер — і коли Docker допомагає, а коли заважає.
4. **Запустити YOLO11** і розуміти різницю між п'ятьма задачами комп'ютерного зору: детекція, сегментація, класифікація, поза, повернуті рамки.

1.3 Що здавати

Усі частини Lab 8 — **індивідуальні**, один PDF на Gradescope.

№	Що	Бали
D1	Три відповіді про DBoW, по 200–250 слів, з прикладами й картинками/схемами	25
D2	Скриншот hello-world у Docker + 200–250 слів про Dockerfile / образ / контейнер / дослідження / коли не варто	5
D3	Пояснення 5 задач YOLO + ваші фото, оброблені всіма 5 задачами (+5 за запуск в офіційному Docker-образі)	15 (+5)
—	Відгук про лабу через Google Form (посилання на Piazza)	+3

Примітка

У репозиторії є пакет `VNAV-labs/lab8` — ROS 2 вузол, що читає результати YOLO. Це стартовий код для **Lab 8.5** (командна), а не для цієї лаби. Коротко про нього — у розділі 5.

2 Deliverable 1 — мішок візуальних слів (DBoW)

Deliverable 1 (25 балів)

Прочитати статтю DBoW і відповісти на 3 питання (кожне 200–250 слів, з конкретними прикладами, можна з картинками): (1) що дає стійкість до зміни освітлення і ракурсу; (2) навіщо IDF у tf-idf і що буде без нього; (3) як розмір словника впливає на швидкість, precision і recall.

2.1 Навіщо це взагалі: проблема впізнавання місця

Уявіть, що ви йдете темним коридором і рахуєте кроки. Після кількох поворотів ви вже не впевнені, де саме знаходитесь — помилки накопичуються. Але щойно ви бачите знайомий плакат на стіні, ви одразу «прив'язуєтесь»: «ага, я біля кабінету 32». Для робота це і є **place recognition** (впізнавання місця), а знайдена відповідність «зараз я там само, де був 5 хвилин тому» — **loop closure**. У GTSAM це просто ще один **BetweenFactor** між двома далекими в часі позами, і він стягує весь накопичений дрейф.

Складність: у робота можуть бути **тисячі** збережених кадрів, а порівнювати треба **в реальному часі**. Порівнювати кожен новий кадр з кожним старим «в лоб» (як ми матчили SIFT у Lab 5) — надто повільно. DBoW розв'язує це трюком з пошукових систем.

2.2 Головна ідея: картинка як текст

Простими словами

Пошуковик не читає кожну сторінку інтернету заново для кожного запиту. Він заздалегідь знає, в яких сторінках трапляється кожне слово. DBoW робить те саме з картинками: перетворює кожну картинку на набір «слів», а далі шукає схожі картинки так, як Google шукає схожі тексти.

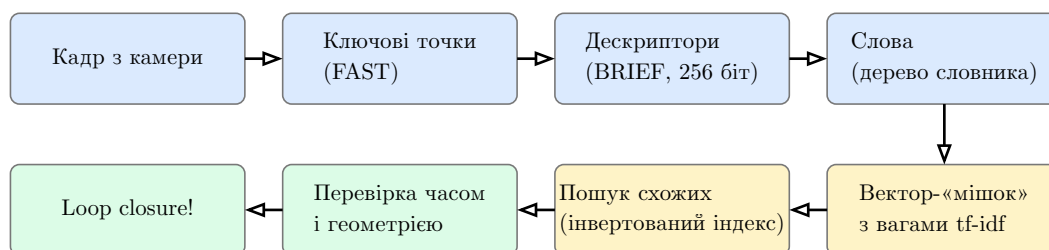


Рисунок 1: Конвеєр DBoW: від кадру до виявленого замикання петлі.

Аналогія, яку варто тримати в голові весь розділ:

	Пошук текстів	DBoW
Документ	сторінка тексту	один кадр з камери
Слово	слово «робот»	«візуальне слово» — тип маленького шматочка картини (кут столу, край рамки, ...)
Словник	усі слова мови	кілька тисяч–мільйон типів шматочків, вивчених заздалегідь
Мішок слів	скільки разів яке слово трапилось (порядок неважливий)	скільки разів яке візуальне слово трапилось на кадрі (де саме — неважливо)

2.3 Як це працює: крок за кроком (як у статті)

1. Ключові точки — FAST. Швидкий детектор «кутів»: точка цікава, якщо на колі навколо неї є довга дуга пікселів, які всі помітно яскравіші або всі темніші за центр. Працює за мікросекунди.

2. Дескриптор — BRIEF. Навколо точки беруть 256 заздалегідь вибраних пар пікселів і для кожної пари питають: «перший яскравіший за другий?» Відповідь «так/ні» — один біт. Разом 256 бітів = 32 байти. Порівнювати два такі дескриптори дуже швидко: рахуємо, у скількох бітах вони відрізняються (відстань Гемінга — одна інструкція процесора).

3. Словник — дерево. Заздалегідь, на великому наборі інших картинок, усі дескриптори кластеризують: ділять на k груп, кожену групу — знову на k , і так L рівнів. Листки дерева — це і є візуальні слова. У статті $k = 10$, $L = 6 \rightarrow 10^6$ слів. Щоб знайти слово для нового дескриптора, достатньо спуститися деревом: на кожному рівні вибрати найближчий із k центрів — це $k \cdot L = 60$ порівнянь замість мільйона.

4. Вектор кадру. Для кадру рахуємо, скільки разів трапилося кожне слово, і зважуємо (tf-idf, див. нижче). Виходить дуже довгий вектор, у якому майже всі числа — нулі.

5. Схожість. Два кадри схожі, якщо їхні вектори схожі: нормуємо обидва вектори (щоб кадр з 300 і кадр з 500 точками були порівнянні) і дивимось, наскільки вони перекриваються. 1 — ідентичні, 0 — жодного спільного слова. Точна формула — у розділі «Глибше».

6. Інвертований індекс — головний прискорювач. Для кожного слова зберігаємо список кадрів, де воно траплялося. Новий кадр дивиться лише у списки своїх слів — кадри без жодного спільного слова навіть не розглядаються. Так пошук серед десятків тисяч кадрів займає мілісекунди.

7. Прямий індекс — для кожного кадру пам'ятаємо, які точки належать до яких слів (на певному рівні дерева). Коли треба перевірити кандидата геометрично, точки порівнюють лише всередині однакових слів — швидше, ніж усі з усіма.

8. Перевірки, щоб не помилитися:

- **нормований бал:** схожість з кандидатом ділять на схожість з попереднім кадром — так поріг не залежить від того, чи сцена «багата» чи «бідна» на текстурі;
- **острівці:** сусідні за часом кандидати групують разом (якщо ми впізнали кадр 500, то 499 і 501 теж схожі — це один «острівець»);
- **часова послідовність:** збіг має підтверджуватись кілька кадрів поспіль;
- **геометрична перевірка:** RANSAC з фундаментальною матрицею (як 8-point у Lab 6) — чи можуть ці точки бути тією самою сценою з іншого ракурсу.



Глибше: математика DBowW

FAST. Беремо піксель p з яскравістю I_p і коло з 16 пікселів радіусу 3 навколо нього. Точка — кут, якщо на колі є неперервна дуга з n пікселів (типово $n = 9$, «FAST-9»), які всі яскравіші за $I_p + t$ або всі темніші за $I_p - t$:

$$\exists \text{ дуга } A, |A| \geq n : \quad \forall x \in A : I_x > I_p + t \quad \text{або} \quad \forall x \in A : I_x < I_p - t. \quad (1)$$

Поріг t регулює, скільки кутів знайдеться. Щоб не отримати купу сусідніх «кутів» на одному місці, лишаяють лише локальні максимуми сили кута (non-maximum suppression).

BRIEF. Навколо точки p беремо патч $S \times S$ пікселів (у статті $S = 48$), згладжуємо Гаусом (щоб один шумний піксель не перевертав біт) і для $L_b = 256$ заздалегідь випадково вибраних пар зсувів (x_i, y_i) рахуємо тест

$$\tau(p; x_i, y_i) = \begin{cases} 1, & I(p + x_i) < I(p + y_i) \\ 0, & \text{інакше} \end{cases}, \quad B(p) = \sum_{i=1}^{L_b} 2^{i-1} \tau(p; x_i, y_i). \quad (2)$$

Зсуви вибирають з гаусового розподілу $\mathcal{N}(0, S^2/25)$ — пари здебільшого ближче до центру патча. Відстань між дескрипторами — Гемінга: $d(B_1, B_2) = \text{popcount}(B_1 \oplus B_2)$ — кількість одиничних бітів у XOR, одна-дві інструкції процесора.

Чому BRIEF стійкий до освітлення — формально. Нехай нова яскравість $I' = \alpha I + \beta$, $\alpha > 0$ (зміна контрасту і яскравості). Тоді $I'(a) < I'(b) \iff \alpha I(a) + \beta < \alpha I(b) + \beta \iff I(a) < I(b)$: **кожен** тест τ дає той самий біт, дескриптор не змінюється взагалі. Те саме для будь-якого монотонного перетворення яскравості (гамма-корекція тощо). Для FAST: якщо t масштабується разом з контрастом, результат теж не змінюється.

Словник. Дескриптори навчального набору кластеризують ієрархічно: **k-medians** (для двійкових векторів «центр» кластера — побітова медіана: біт 1, якщо в більшості дескрипторів кластера цей біт 1) з ініціалізацією k-means++ (перші центри вибирають далеко один від одного). На кожному з L рівнів — k гілок, листків $W = k^L$. Пошук слова для дескриптора: на кожному рівні k відстаней Гемінга $\rightarrow kL$ операцій (для $k = 10$, $L = 6$: 60 замість 10^6).

Вектор кадру і схожість. Кадр $I_t \rightarrow$ вектор $v_t \in \mathbb{R}^W$ з вагою $w_i = \text{tf-idf}$ (формула в питанні 2) для кожного слова. Схожість — L_1 -оцінка:

$$s(v_1, v_2) = 1 - \frac{1}{2} \left\| \frac{v_1}{\|v_1\|_1} - \frac{v_2}{\|v_2\|_1} \right\|_1 \in [0, 1]. \quad (3)$$

L_1 , а не косинус чи L_2 : для розріджених BoW-векторів вона експериментально точніша (Nistér & Stewénius, 2006) і рахується лише по словах, що є хоча б в одному з векторів.

Нормований бал. Абсолютне значення s залежить від сцени: у «бідній» на текстурі сцені навіть сусідні кадри мають низьку схожість. Тому бал ділять на схожість з попереднім кадром:

$$\eta(v_t, v_{t_j}) = \frac{s(v_t, v_{t_j})}{s(v_t, v_{t-\Delta t})}. \quad (4)$$

«Наскільки кандидат схожий на мене, порівняно з тим, наскільки я схожий на себе секунду тому». Кандидатів з $\eta < \alpha$ відкидають; якщо сам знаменник замалий (кадр розмитий, нема на що спертися), кадр пропускають.

Острівці. Кандидати, близькі за часом $(t_{n_i}, \dots, t_{m_i})$, об'єднують в острівця V_{T_i} з балом $H(v_t, V_{T_i}) = \sum_{j=n_i}^{m_i} \eta(v_t, v_{t_j})$. Вибирають острівця з найбільшим H . Це прибирає ситуацію, коли кадри 499, 500, 501 «змагаються» між собою за звання збігу.

Часова послідовність. Збіг приймають, лише якщо для k попередніх запитів (у статті $k = 3$) найкращі острівці теж перекривалися з поточним — справжнє повернення в місце триває кілька кадрів, випадковий збіг — ні.

Геометрична перевірка. Для відповідностей між точками поточного кадру і кадру-кандидата оцінюють **фундаментальну матрицю** F ($x_2^T F x_1 = 0$ — епіполярне обмеження з Lab 6, але для некаліброваної камери) з RANSAC; потрібно щонайменше 12 inliers. **Прямий індекс** прискорює це: точки порівнюють лише всередині однакових вузлів дерева на певному рівні l (параметр статті), а не всі з усіма.

Precision і recall (знадобляться в питанні 3):

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad (5)$$

де TP — правильно знайдені замикання петлі, FP — хибні (різні місця прийняті за одне), FN — пропущені (повернулися, але не впізнали). Для SLAM важливіша **100% precision**: один

хибний loop closure ламає карту, а пропущений лише трохи затримує корекцію дрейфу. У статті параметри підбирають саме так: максимальний recall при precision = 100%.

2.4 Питання 1 — стійкість до освітлення і ракурсу

Що має бути у відповіді: назвати конкретні компоненти й пояснити, чому кожен допомагає.

- **Освітлення.** FAST і BRIEF порівнюють яскравості пікселів **між собою**, а не абсолютні значення. Якщо кімнату освітити сильніше, усі пікселі стануть яскравішими, але «лівий яскравіший за правий» залишиться правдою. Тож біти BRIEF не зміняться. Приклад: той самий коридор вранці й увечері.
- **Квантування у слова.** Дескриптор трохи змінився від шуму чи відблиску — але він, найімовірніше, досі найближчий до того самого центру в дереві, тож слово те саме. Словник «поглинає» дрібні відмінності.
- **Мішок без порядку.** Вектор не знає, **де** на кадрі було слово. Якщо ви стали на метр лівіше, об'єкти зсунулися в кадрі, але набір слів майже той самий → схожість висока.
- **Нормування вектора** (ділення на L_1 -норму): частина точок зникла через ракурс чи перекриття — вага решти перерахується, кадр досі схожий.
- **Геометрична і часова перевірка** не додають стійкості до ракурсу, а навпаки, відсіюють **хибні** збіги, які стійкість могла пропустити.

Чесна межа, яку варто згадати: BRIEF у статті **не** інваріантний до повороту в площині кадру і до масштабу. Для робота, що їздить по підлозі з камерою вперед, це нормально; для дрона, що нахиляється, — ні. Тому ORB-SLAM використовує ORB (BRIEF з орієнтацією) — хороший приклад для відповіді.

Порада

Картинка для відповіді: одна сцена в двох освітленнях з однаковими кольоровими кружечками на точках, що дали те саме слово. Можна намалювати схему «пара пікселів → біт» і показати, що при +50 до яскравості біт не змінився.

2.5 Питання 2 — навіщо IDF

Термін **tf-idf** — це вага слова в кадрі:

$$w_i = \underbrace{\frac{n_{id}}{n_d}}_{\text{tf}} \cdot \underbrace{\log \frac{N}{N_i}}_{\text{idf}}. \quad (6)$$

- **tf** (term frequency): яку частку точок кадру займає слово i (n_{id} — скільки разів слово i на кадрі d , n_d — усього слів на кадрі);
- **idf** (inverse document frequency): наскільки слово **рідкісне** серед усіх N кадрів (N_i — у кількох кадрах воно взагалі траплялось).

Простими словами

Якщо слово є майже скрізь, воно нічого не каже про місце. У тексті це «і», «в», «на»: ними неможливо відрізнити статтю про роботів від рецепту борщу. На картинках — плитка підлоги, текстура стелі, однакові двері в офісі. IDF автоматично «приглушує» такі слова ($N_i \approx N \Rightarrow \log N/N_i \approx 0$) і «підсилює» рідкісні — унікальний постер, незвичний предмет.

Що буде без IDF: схожість двох кадрів визначатимуть найчастіші, найбанальніші слова. Два **різні** коридори з однаковою плиткою виглядатимуть майже ідентично → **хибні loop closure (perceptual aliasing)**. А хибне замикання петлі в SLAM — катастрофа: GTSAM «зшиє» дві різні точки простору і зламає всю карту. Також рідкісні, справді інформативні слова тонути серед частих — справжні збіги матимуть нижчий бал.

Приклад для відповіді: офіс з 20 однаковими дверима й одним яскравим плакатом. Без IDF кадри з дверима з різних поверхів «схожі»; з IDF вирішальним стає плакат.



Глибше: IDF у числах

Нехай у базі $N = 1000$ кадрів. Слово «плитка підлоги» трапилось у $N_i = 900$ кадрах: $\text{idf} = \ln(1000/900) \approx 0.105$. Слово «край цього плаката» — у 5 кадрах: $\text{idf} = \ln(1000/5) \approx 5.3$. Рідкісне слово важить у **50 разів** більше. Слово, що є в **кожному** кадрі, має $\text{idf} = \ln 1 = 0$ — повністю ігнорується, як стоп-слова в пошуковику. Без IDF обидва слова мали б вагу, пропорційну лише частоті на кадрі, а плитки на кадрі зазвичай **більше**, ніж плаката.

2.6 Питання 3 — розмір словника

Простими словами

Словник — це наскільки «дрібно» ми ділимо всі можливі шматочки картинок на типи. Мало слів — грубий поділ («щось кутасте»). Багато слів — дуже дрібний («саме такий кут саме такої рамки»).

	Малий словник	Великий словник
Обчислення	дешево будувати й зберігати; але списки інвертованого індексу довгі — кожне слово трапляється в багатьох кадрах, пошук повільніший	дорожче навчання й пам'ять; перетворення дескриптора в слово — лише $O(k \cdot L)$, росте як логарифм від розміру; списки індексу короткі → пошук швидший
Precision	низька: різні шматочки потрапляють в одне слово → різні місця виглядають схожими	висока: слово дуже специфічне, випадковий збіг малоімовірний
Recall	висока: той самий об'єкт майже завжди дає те саме слово, навіть з шумом	падає: через шум чи ракурс той самий шматочок може потрапити в сусіднє слово → справжній збіг пропущено

Висновок: більше слів — не завжди краще. Є «золота середина», яку підбирають експериментально на даних; у статті це 10^6 слів ($k = 10$, $L = 6$), навчених на окремому наборі зображень. Ще один нюанс: словник, навчений на одних сценах (вулиця), гірше працює на інших (приміщення) — слова «не ті».

Приклад для відповіді: порахуйте для малого ($k = 10$, $L = 2 \rightarrow 100$ слів) і великого (10^6) словника, скільки порівнянь потрібно на один дескриптор (20 проти 60) і скільки кадрів у середньому потрапить у список одного слова.



Глибше: чому recall падає зі зростанням словника

Спуск деревом — це L рішень поспіль. Нехай шум (інше освітлення, ракурс) з імовірністю q не змінює вибір гілки на одному рівні. Тоді той самий фізичний шматочок сцени потрапить у те саме слово з імовірністю приблизно q^L — **експоненційно** менше з кожним новим рівнем. При $q = 0.95$: $L = 2 \rightarrow 0.90$, $L = 6 \rightarrow 0.74$, $L = 10 \rightarrow 0.60$. Так recall окремих слів падає, хоча кожне слово стає специфічнішим.

Довжина списків інвертованого індексу. Якщо в кадрі в середньому m різних слів, а слова розподілені рівномірно, то одне слово трапляється приблизно в Nm/W кадрах. Для $N = 10^4$ кадрів, $m = 300$: $W = 100 \rightarrow 3 \cdot 10^4$ записів (по суті кожен кадр у кожному списку — індекс

марний), $W = 10^6 \rightarrow 3$ записи. Тому великий словник робить **пошук** швидшим, хоча дерево глибше.

Пастка

Курс прямо просить: відповіді — **своїми словами**. Гайд дає думки й приклади; складіть з них власний текст із власними прикладами й схемами.

3 Deliverable 2 — Docker

Deliverable 2 (5 балів)

Встановити Docker, прикласти скриншот `hello-world`. У 200–250 словах: що таке Dockerfile, образ, контейнер; як використати Docker у дослідницькій публікації і чому це краще за інші способи; коли Docker не варто використовувати.

3.1 Простими словами

Простими словами

Docker — це спосіб запакувати програму **разом з усім, що їй потрібно**: потрібна версія Ubuntu, ROS, бібліотеки, налаштування. Така «коробка» запускається однаково на будь-якому комп'ютері з Docker. Кінець епохи «у мене працює, а в тебе — ні».

Аналогія з кухнею:

Поняття	Кухня	У нашому репозиторії
Dockerfile	рецепт: які інгредієнти й кроки	ros2-docker/Dockerfile: база ros:humble, apt-пакети, OpenGV, GTSAM, ultralytics...
Образ (image)	готова заготовка за рецептом; незмінна, можна зробити скільки завгодно порцій	ros2-vnav:humble (7.7 ГБ) — зібраний один раз
Контейнер (container)	конкретна порція, яку їсть конкретна людина; можна змінити, а заготовка лишається та сама	ros2-vnav-lab7-dev — запущений з образу, в нього змонтована ваша тека з кодом

Важлива деталь: образ складається з **шарів** — кожен рядок `RUN` у Dockerfile дає шар. Якщо змінити останній рядок, попередні шари беруться з кешу. Ми це бачили: після правки кроку з PyTorch перезбирання тривало 4 хвилини замість години.

Глибше: як Docker влаштований усередині

Контейнер — не віртуальна машина. Віртуальна машина емулює цілий комп'ютер зі своїм ядром ОС. Контейнер — це звичайний процес на ядрі **хоста**, якого ядро Linux «обманює»:

- **namespaces** — процес бачить свою файлову систему, свої PID (у контейнері він PID 1), свою мережу, своє ім'я хоста;
- **cgroups** — ядро обмежує йому CPU і пам'ять.

Тому контейнер стартує за частки секунди і майже не втрачає швидкості.

Шари й кеш. Кожна інструкція Dockerfile (`RUN`, `COPY`, ...) дає незмінний шар — архів змін файлової системи. Шар ідентифікується хешем: інструкція + попередні шари (+ вміст файлів для `COPY`). Якщо хеш збігся — шар береться з кешу. Звідси правило: рідко змінювані кроки (`apt`) — вгорі, часто змінювані — внизу.

Copy-on-write. Контейнер = шари образу (лише читання) + тонкий шар для запису зверху (`overlayfs`). Змінили файл образу — його копія лягає у верхній шар; образ не змінюється. `docker run --rm` після завершення видаляє цей верхній шар. Тому все, що треба зберегти, кладуть у **том** (`-v`) — теку хоста, змонтовану всередину.

Чому на Mac складніше. Контейнерам потрібне ядро Linux, тож Docker Desktop на Mac запускає приховану Linux-віртуалку, а контейнери — в ній. Звідси наші проблеми: GUI йде через мережевий X11 до XQuartz (без нормального OpenGL), вебкамера Mac не «прокидається» в Linux-віртуалку, а x86-образ на ARM-процесорі виконується через емуляцію (Rosetta/QEMU) — в рази повільніше.

3.2 Встановлення і скриншот

- **Ubuntu:** офіційна інструкція, розділ «Install using the apt repository»: додати ключ і репозиторій Docker, `sudo apt install docker-ce docker-ce-cli containerd.io`. Посилання — у завданні.
- **macOS:** Docker Desktop (у вас уже встановлений).

Скриншот:

```
docker run hello-world
```

Має з'явитися «Hello from Docker! This message shows that your installation appears to be working correctly.»

3.3 Думки для відповідей

Дослідження й публікації. Автор статті кладе в репозиторій Dockerfile (або публікує образ). Будь-хто може відтворити результати однією командою, з **тими самими** версіями бібліотек — навіть через 5 років, коли ці версії вже не встановлюються звичайним способом. Це краще за README «встановіть OpenCV 3.4 і ROS Melodic»: такі інструкції застарівають, конфліктують з іншим софтом на комп'ютері, і їх часто неможливо виконати. Приклад з курсу: Lab 9 запускає ORB-SLAM3 і Kimera в готових образах — без них збирання цих систем займало б дні.

Коли Docker не варто (приклади з нашого досвіду — хороший матеріал для відповіді):

- **Графіка і GUI.** RViz2 у Docker на Mac не зміг створити OpenGL-вікно через XQuartz — довелося перейти на Foxglove.
- **Доступ до заліза.** Вебкамера `/dev/video0`, GPU (CUDA), USB-сенсори на роботі — у Docker Desktop на Mac недоступні або потребують складних налаштувань.
- **Продуктивність.** Образ курсу — x86, на Apple Silicon він працює через емуляцію: SIFT у Lab 6 обробляв 7 кадрів/с замість 20.
- **Розмір.** Образ легко роздувається: PyTorch з CUDA додав 5 ГБ, які на Mac були непотрібні.
- **Реальний час і вбудовані системи** — на дроні з обмеженими ресурсами зайвий шар ізоляції небажаний.
- **Дуже прості проєкти** — один Python-скрипт простіше запустити в `venv`.

4 Deliverable 3 — YOLO з Ultralytics

Deliverable 3 (15 (+5) балів)

Пояснити своїми словами 5 задач: detection, segmentation, classification, pose, oriented bounding boxes. Встановити YOLO11 (Ultralytics), обробити **власні** фото всіма 5 задачами (не `bus.jpg` з документації!), вставити результати в PDF. Бонус: запустити YOLO в офіційному Docker-образі й пояснити, як передати фото всередину і забрати результат.

4.1 Що таке YOLO, простими словами

Простими словами

YOLO — «You Only Look Once», «дивившись лише раз». Старі детектори спочатку шукали «підозрілі» області на картинці, а потім окремо розглядали кожну. YOLO дивиться на всю картинку **одним проходом нейромережі** і відразу видає: «тут людина з упевненістю 91%, тут автобус — 87%». Тому він достатньо швидкий для відео в реальному часі — саме те, що треба роботі.

Що відбувається всередині (без деталей мережі):

1. картинку зменшують до стандартного розміру (наприклад, 640 пікселів);
2. нейромережа для кожної ділянки картинки передбачає: чи є тут об'єкт, яка його рамка і клас;
3. відкидаються слабкі передбачення (низька впевненість) і дублювати тієї самої рамки (**NMS** — non-maximum suppression: з кількох рамок одного об'єкта лишається найвпевненіша);
4. на виході — список об'єктів: клас, упевненість, рамка (або маска, або ключові точки — залежно від задачі).

Моделі YOLO11 бувають різного розміру: n (nano, найшвидша) $\rightarrow s \rightarrow m \rightarrow l \rightarrow x$ (найточніша, найповільніша). На CPU беріть n .

4.2 П'ять задач

Задача	Що видає, простими словами	Модель
Detection (детекція)	прямокутник навколо кожного об'єкта + клас + упевненість. «Тут людина, тут автобус.»	yolo11n.pt
Segmentation (сегментація)	для кожного об'єкта — точний контур (маска пікселів), а не прямокутник. Видно, які саме пікселі належать людині.	yolo11n-seg.pt
Classification (класифікація)	одна мітка на всю картинку, без місця: «на цьому фото — мікроавтобус». Об'єктів не шукає.	yolo11n-cls.pt
Pose (поза)	для кожної людини — 17 ключових точок (ніс, очі, плечі, лікті, коліна...) і «скелет» між ними. Видно, як людина стоїть.	yolo11n-pose.pt
ОВВ (повернуті рамки)	прямокутники, повернуті під кутом об'єкта. Для знімків згори (супутник, дрон), де кораблі й машини стоять під різними кутами.	yolo11n-obb.pt

Нижче — справжні результати, отримані командами з розділу 4.4 у Docker-образі курсу (на CPU, 0.1–0.8 с на зображення). Це **демонстрація**: для задачі потрібні ваші власні фото.

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \in [0, 1]. \quad (7)$$

NMS (non-maximum suppression) прибирає дублікати: сортуємо кандидати за впевненістю; беремо найкращого; видаляємо всіх того самого класу з $\text{IoU} > \text{iou}$ з ним (за замовчуванням $\text{iou} = 0.7$); повторюємо з рештою. Результат — по одній рамці на об'єкт.

Сегментація. Мережа додатково видає 32 «базові маски» (прототипи) P_k розміром 160×160 на всю картинку, а для кожного об'єкта — 32 коефіцієнти c_k . Маска об'єкта — їх зважена сума, пропущена через сигмоїду й обрізана рамкою:

$$M = \sigma \left(\sum_{k=1}^{32} c_k P_k \right). \quad (8)$$

Так мережа не малює окрему маску для кожного з 8400 кандидатів — дешево.

Класифікація. Одна голова на всю картинку: вектор «оцінок» z_i для 1000 класів ImageNet $\rightarrow$ softmax $p_i = e^{z_i} / \sum_j e^{z_j}$, $\sum p_i = 1$. Виводиться топ-5 класів — як у нашому прикладі («minibus 0.57, police_van 0.34, ...»).

Поза. Для кожної рамки-людини — 17 точок COCO (ніс, очі, вуха, плечі, лікті, зап'ястя, стегна, коліна, щиколотки), для кожної — (x, y) і «видимість». Якість вимірюють **OXS** — аналогом IoU для точок: $\text{OXS} = \exp(-d^2 / (2s^2\kappa^2))$, де d — відстань до правильної точки, s — розмір людини, κ — допуск для типу точки (для зап'ястя більший, ніж для ока).

ОВВ. Рамка задається п'ятьма числами (x, y, w, h, θ) — з кутом повороту. IoU повернутих прямокутників рахувати незручно, тож при навчанні рамку представляють гаусовим розподілом і порівнюють розподіли (ProbIoU). Модель навчена на DOTA — 15 класів аерооб'єктів (кораблі, літаки, машини, гавані, мости, спортмайданчики...).

Як міряють якість детектора. **mAP** (mean Average Precision): для кожного класу будують криву precision-recall при різних порогах впевненості, площа під нею — AP; усереднюють по класах. «mAP50-95» — ще й усереднено по порогах IoU від 0.5 до 0.95 (рамка вважається правильною, лише якщо перекривається достатньо). Ті самі precision і recall, що й у DBoW.

4.3 Як вибрати фото (найчастіша причина втрачених балів)

- **Detection / Segmentation:** кілька різних предметів з 80 класів COCO — люди, чашки, ноутбуки, стільці, машини, тварини.
- **Classification:** фото, де в кадрі **один** головний об'єкт великим планом (тварина, фрукт, предмет).
- **Pose:** люди на повний зріст, бажано в різних позах (хтось сидить, хтось піднімає руку).
- **ОВВ:** вигляд **згори**. Модель навчена на аерофотознімках (набір DOTA: кораблі, літаки, машини, резервуари, спортмайданчики). Звичайне фото збоку не спрацює. Найпростіше — скріншот супутникової карти парковки чи порту.

4.4 Встановлення і запуск

Варіант А — прямо на Mac (найпростіше і найшвидше):

```
python3 -m pip install --user ultralytics
export PATH="$PATH:$HOME/.local/bin" # якщо команда yolo "not found"
cd ~/Pictures/lab8 # тека з вашими фото
yolo detect predict model=yololln.pt source=photo_people.jpg
yolo segment predict model=yololln-seg.pt source=photo_people.jpg
yolo classify predict model=yololln-cls.pt source=photo_cat.jpg
yolo pose predict model=yololln-pose.pt source=photo_people.jpg
yolo obb predict model=yololln-obb.pt source=aerial_parking.jpg
```

На Apple Silicon можна додати `device=mps` — рахує на GPU Mac.

Варіант Б — у Docker-образі курсу (ultralytics уже встановлений; саме так отримано приклади вище):

```
cd ~/Pictures/lab8
docker run --rm --platform linux/amd64 -v "$PWD:/data" -w /data \
  -e YOLO_CONFIG_DIR=/tmp/yolo ros2-vnav:humble \
  yolo detect predict model=yololln.pt source=photo_people.jpg
```

Де **результат**: кожна команда друкує рядок `Results saved to ...` — там лежить картинка з намальованими рамками/масками. За замовчуванням це `runs/<задача>/predict`, наступний запуск — `predict2` тощо. Ваги моделей (`yololln*.pt`, 3–6 МБ) завантажуються автоматично при першому запуску.

Пастка

`source=~/photo.jpg` не спрацює: YOLO не розгортає `~`. Пишіть відносний (`source=photo.jpg`) або повний шлях (`source=/Users/you/Pictures/photo.jpg`).

4.5 Бонус: офіційний Docker-образ Ultralytics

Питання бонусу: **як передати фото в контейнер і забрати результат?** Відповідь — **том** (volume): прапорець `-v <тека на комп'ютері>:<тека в контейнері>` робить теку спільною. Контейнер читає звідти фото і пише туди `runs/...`, які після завершення лишаються у вас на диску.

```
cd ~/Pictures/lab8
docker run --rm -it -v "$PWD:/data" -w /data ultralytics/ultralytics:latest-arm64 \
  yolo detect predict model=yololln.pt source=photo_people.jpg
# результат: ~/Pictures/lab8/runs/detect/predict/photo_people.jpg
```

`latest-arm64` — образ для Apple Silicon (на Linux/x86 — `ultralytics/ultralytics:latest-cpu` або `latest`). Цей офіційний образ важить кілька гігабайтів; я його не завантажував, тож теж варто звірити з Docker Hub. Той самий принцип з томом показано у варіанті Б вище — і він перевірений.

5 Міст до Lab 8.5: пакет `lab8` у репозиторії

У Lab 8.5 YOLO працює не на окремих фото, а як частина ROS 2 системи: знаходить ведмедика в кожному кадрі відео (набір TUM freiburg3_teddy), а ви за центрами його рамок і позами камери будувате факторний граф у GTSAM і **триангулюєте** 3D-позицію ведмедика. Тобто поєднання Lab 7 (GTSAM) і Lab 8 (YOLO).

Стартовий код уже лежить у `VNAV-labs/lab8`:

- `src/deliverable_1.cpp` — вузол, що підписується на `/yolo_result` (повідомлення `ultralytics_ros`) і для кожної детекції друкує клас, упевненість і центр рамки в пікселях, а також позу камери з `/tf` (`world` → `openni_rgb_frame`);
- `launch/deliverable_1.launch.yaml` — вебкамера (`usb_cam`) + YOLO (`ultralytics_ros`) + цей вузол.

Примітка

На Mac Docker не бачить вебкамеру (`/dev/video0`), а `deliverable_1.launch.yaml` завжди стартує `usb_cam`. Для 8.5 запускайте частини окремо, як у завданні: `ros2 bag play -l <датасет>`, `ros2 launch ultralytics_ros tracker.launch.xml input_topic:=/camera/rgb/image_color` і `ros2 run lab_8 deliverable_1`. Докладний гайд до 8.5 — окремо.

6 Чекліст здачі

- **[?] D1:** три відповіді по 200–250 слів — стійкість (освітлення + ракурс, з назвами компонентів), IDF (з прикладом «однакових дверей»), розмір словника (обчислення, precision, recall); принаймні одна схема чи картинка.
- **[?] D2:** скриншот `docker run hello-world`; відповіді про Dockerfile, образ, контейнер, дослідження, коли не варто.
- **[?] D3:** пояснення 5 задач; **ваші** фото, оброблені кожною задачею, з підписом «яке фото — яка задача»; фото для pose — з людьми, для OBB — вигляд згори.
- **[?] (+5)** запуск в офіційному Docker-образі: скриншот + як фото потрапило всередину (том - v) і як результат вийшов назовні.
- **[?] (+3)** відгук через Google Form (посилання на Piazza).

А Словничок

Термін	Простими словами
Loop closure	«я тут уже був» — зв'язок між поточним і давнім кадром, що прибирає накопичений дрейф
Place recognition	задача впізнати місце за картинкою
Perceptual aliasing	різні місця виглядають однаково (однакові коридори) — головне джерело хибних збігів
FAST	швидкий детектор «кутів» на картинці
BRIEF	дескриптор з 256 бітів «яскравіше/темніше» для пар пікселів
Відстань Гемінга	у скількох бітах відрізняються два двійкові рядки
Візуальне слово	тип маленького шматочка картинки; листок дерева словника
Словник / дерево словника	усі візуальні слова, організовані в дерево для швидкого пошуку
tf	як часто слово трапляється в цьому кадрі
idf	наскільки слово рідкісне серед усіх кадрів; часті слова отримують малу вагу
Інвертований індекс	для кожного слова — список кадрів, де воно є
Прямий індекс	для кожного кадру — які його точки до яких слів належать
Precision	яка частка знайдених збігів справжня
Recall	яку частку справжніх збігів ми знайшли
Dockerfile / образ / контейнер	рецепт / готова заготовка / конкретний запущений екземпляр
Том (volume, -v)	спільна тека між комп'ютером і контейнером
YOLO	нейромережа, що за один прохід знаходить об'єкти на картинці
NMS	прибирання дублікатів: з кількох рамок одного об'єкта лишається найвпевненіша
COCO / ImageNet / DOTA	набори даних: 80 класів для детекції / 1000 для класифікації / аерознімки для OBB

Джерела: завдання курсу VNAV (MIT, CC BY 4.0); D. Gálvez-López, J. D. Tardós, IEEE T-RO 2012; документація Ultralytics; офіційна документація Docker.